

# NumPy Beginner Level Complete Notes

## Introduction

NumPy is a Python library used for numerical computing and array operations.  
Key idea: NumPy arrays are faster and more memory efficient than Python lists.

## Array Creation Functions

`np.array()` - Convert Python data into a NumPy array.

Example: `np.array([1,2,3])`

`np.zeros(shape)` - Create an array filled with 0.

Example: `np.zeros((2,3))`

`np.ones(shape)` - Create an array filled with 1.

Example: `np.ones((2,3))`

`np.full(shape, value)` - Create an array filled with a chosen value.

Example: `np.full((2,3), 7)`

`np.empty(shape)` - Create an array without initializing values.

Values are unpredictable.

`np.arange(start, stop, step)` - Create evenly spaced values using a step size.

Example: `np.arange(1,11)`

`np.linspace(start, stop, num)` - Create a fixed number of evenly spaced values.

Example: `np.linspace(0,100,5)`

## Array Attributes

`arr.ndim` - Number of dimensions.

`arr.shape` - Size of each dimension.

Examples:

`(5,)` -> 1D with 5 elements

`(2,3)` -> 2 rows, 3 columns

`(2,3,4)` -> 2 blocks, 3 rows, 4 columns

`arr.size` - Total number of elements.

Formula: product of shape values.

`arr.dtype` - Data type stored in array.

Examples: `int64`, `float64`, `bool`

`arr.itemsize` - Memory used by one element in bytes.

Typical values:

`int64` -> 8 bytes

`float64` -> 8 bytes

`bool` -> 1 byte

# Indexing

1D Indexing:

`arr[index]`

Positive indexing starts from 0.

Negative indexing starts from -1 (last element).

2D Indexing:

`arr[row, column]`

3D Indexing:

`arr[block, row, column]`

Negative indexing works in every dimension.

Example:

`arr[-1,-1]` -> last row, last column

`arr[-1,-1,-1]` -> last block, last row, last column

## 1D Slicing

Syntax:

`arr[start:stop]`

start included, stop excluded.

Examples:

`arr[1:4]`

`arr[:3]`

`arr[3:]`

`arr[:]`

Step slicing:

`arr[start:stop:step]`

Examples:

`arr[:,2]`

`arr[1:6:2]`

Reverse:

`arr[::-1]`

`arr[::-2]`

## 2D Slicing

Syntax:

`arr[row_slice, column_slice]`

Examples:

`arr[0,:]` -> first row

`arr[-1,:]` -> last row

`arr[:,0]` -> first column

`arr[:,-1]` -> last column

Submatrix:

```
arr[0:2,0:2]
```

Reverse columns:

```
arr[:,::-1]
```

Reverse rows:

```
arr[::-1,:]
```

## Reshaping

```
reshape(new_shape)
```

Changes shape without changing data.

Rule:

Product of new shape must equal total elements.

Examples:

```
arr.reshape(2,3)
```

```
arr.reshape(3,2)
```

Automatic dimension:

```
arr.reshape(2,-1)
```

Used heavily in Machine Learning.

## Flatten and Ravel

```
flatten()
```

Converts array to 1D.

Returns a copy.

```
ravel()
```

Converts array to 1D.

Usually returns a view.

Changes may affect original array.

`flatten()` -> safer

`ravel()` -> faster

## Transpose

```
transpose()
```

Rows become columns and columns become rows.

Example:

```
arr.transpose()
```

Shortcut:

```
arr.T
```

Shape changes:

(2,3) -> (3,2)

# Important Rules and Memory Tricks

ndim = How many dimensions?  
shape = How big is each dimension?  
size = Total elements  
dtype = Data type  
itemsize = Memory per element

Indexing = one value  
Slicing = multiple values

arange() -> fixed step size  
linspace() -> fixed number of values

reshape() rule:  
New shape product = total elements